

Contents

1	DD07: 通知ロジック	1
1.1	0. 文書情報	1
1.2	1. 対象範囲	1
1.3	2. 収録ロジック・対応章	1
1.4	3. 詳細設計本文	2
1.4.1	3.1 EmailProvider 抽象	2
1.4.2	3.2 enqueueEmail	2
1.4.3	3.3 Consumer 再試行	2
1.4.4	3.4 Resend Webhook 処理	2
1.4.5	3.5 サプレスチェック	3
1.4.6	3.6 通知重要度と強制送信	3
1.4.7	3.7 メール本文の表示義務（特定電子メール法）	3
1.5	4. 関連設計	3
1.6	5. テスト観点	4
1.6.1	5.1 その他観点	4
1.7	6. 未確定事項・確認事項	4

1 DD07: 通知ロジック

1.1 0. 文書情報

項目	内容
文書名	DD07: 通知ロジック
詳細設計 ID	DD07
対象システム	FAQ AI ウィジェット SaaS / メインシステム
関連機能 ID	FR-140～149（通知 14 種 / Resend / Webhook） / AC-021
作成日	2026-05-17
版数	v1.0
ステータス	承認済

1.2 1. 対象範囲

種別	ID	名称
機能	FR-140～149	通知 14 種 / メール送信 / Webhook 受信 / サプレッション
API	/projects/{id}/notification-settings	(v2.0 で廃止)SCR-030 廃止、プロジェクト関連通知は常時 ON 固定
API	/webhooks/resend	Resend Webhook 受信
テーブル	notification_logs email_suppression_list unknown_webhook_events	通知基盤
Queue	email-queue	メール送信 Queue

1.3 2. 収録ロジック・対応章

元章	元タイトル	概要
§ 10.4.1	EmailProvider 抽象	TypeScript インターフェース + Resend 実装
§ 10.4.2	enqueueEmail	Queue 投入（基本設計参照）
§ 10.4.3	Consumer 再試行	指数 BO（60s, 120s, 240s, ..., max 600s）、最大 3 回
§ 10.4.4	Resend Webhook 処理	8 イベント分岐 + 未知イベント記録
§ 10.4.5	サプレスチェック	送信前 HMAC 照合

1.4 3. 詳細設計本文

1.4.1 3.1 EmailProvider 抽象

```
// app/shared/src/adapters/email-provider.ts
export interface EmailProvider {
  send(input: EmailInput): Promise<{ messageId: string; provider: string }>;
}

export type EmailInput = {
  from: string;
  to: string;
  subject: string;
  html: string;
  text: string;
  replyTo?: string;
  tags?: Record<string, string>;
  idempotencyKey?: string;
};
```

1.4.1.1 3.1.1 ResendEmailProvider

```
// app/workers/queue-consumer/src/adapters/resend-email-provider.ts
export class ResendEmailProvider implements EmailProvider {
  constructor(private env: Env) {}
  async send(input: EmailInput) {
    const res = await fetch('https://api.resend.com/emails', {
      method: 'POST',
      headers: {
        'Authorization': `Bearer ${this.env.RESEND_API_KEY}`,
        'Content-Type': 'application/json',
        ...(input.idempotencyKey ? { 'Idempotency-Key': input.idempotencyKey } : {}),
      },
      body: JSON.stringify({
        from: input.from, to: input.to, subject: input.subject,
        html: input.html, text: input.text, reply_to: input.replyTo, tags: input.tags,
      }),
    });
    if (!res.ok) throw new Error(`resend_error:${res.status}:${await res.text()}`);
    const data = await res.json() as { id: string };
    return { messageId: data.id, provider: 'resend' };
  }
}
```

1.4.2 3.2 enqueueEmail

通知契機ごとに `enqueueEmail()` を呼び出し、`email-queue` に Queue 投入する。詳細は [../02_基本設計/02_API設計.md](#) を参照。

1.4.3 3.3 Consumer 再試行

指数バックオフ: 60s, 120s, 240s, ... (最大 600s)。3 回失敗で DLQ。詳細は [DD14_ バッチ・非同期処理.md](#) §14.3 を参照。

1.4.4 3.4 Resend Webhook 処理

```
// app/workers/webhook/src/routes/resend.ts
app.post('/webhooks/resend', verifyResendSignature, idempotency, async (c) => {
  const event = await c.req.json<ResendEvent>();
  switch (event.type) {
    case 'email.sent': return handleSent(event, c.env);
    case 'email.delivered': return handleDelivered(event, c.env);
  }
});
```

```

case 'email.bounced':      return handleBounced(event, c.env);
case 'email.complained':    return handleComplained(event, c.env);
case 'email.delivery_delayed': return handleDelayed(event, c.env);
case 'email.opened':        return handleOpened(event, c.env);
case 'email.clicked':       return handleClicked(event, c.env);
case 'email.failed':        return handleFailed(event, c.env);
default:
  await c.env.DB.prepare(
    `INSERT INTO unknown_webhook_events (provider, event_type, payload, received_at)
    VALUES ('resend', ?1, ?2, ?3)`
  ).bind(event.type, JSON.stringify(event), new Date().toISOString()).run();
  return c.json({ ok: true });
}
});

```

bounced のうち soft bounced は 5 連続で permanent suppression。complained は即時 permanent。

1.4.5 3.5 サプレスチェック

```

// 送信前
const emailHmac = await hmacSha256(env.MASTER_KEY, to.toLowerCase());
const suppressed = await env.DB.prepare(
  `SELECT 1 FROM email_suppression_list WHERE email_hmac = ?1 AND is_permanent = 1 AND released`
).bind(emailHmac).first();
if (suppressed) {
  await markSuppressed(env, messageId);
  return;
}

```

1.4.6 3.6 通知重要度と強制送信

通知重要度は `low/normal/high/critical` の 4 区分（共有概念）。`critical` はメール送信が必須となるイベント (`account_project_grants.role='admin' AND valid=1` を 1 件でも保持する全アカウント = オーナー自身も `admin` 行を保持するため自動的に網羅) に予約。`critical` はシステム判定で強制送信される。

critical 通知の宛先解決クエリ: オーナーが自動 `admin` 行を持つため、`account_project_grants.role='admin' AND valid=1` の `EXISTS` 単独条件で宛先を網羅できる。`SELECT DISTINCT account_id` でメンバーが複数 PJ で `admin` を保持する場合の二重カウントを防ぐ。

プロジェクト関連通知 (`NOTIF-CHAT_REPLY_TO_STAFF / NOTIF-CHAT_HOLD_CHECK` 等の `normal / low`) は常時 ON 固定とし、配信時点で `account_project_grants` を `WHERE valid=1` 条件で動的スキャンして当該プロジェクトの全メンバー (オーナー + `admin` + `member`) に即時配信する (画面設定 SCR-030 / `project_notification_settings` テーブルは持たない)。契約単位の `normal / high` 通知 (課金・契約・運営お知らせ等) は契約 WS 側で扱う。個人通知オプトアウト・メンバー選択型 UI は MVP では提供しない。

`announcement.kind = critical` は障害告知や緊急法令対応など、配信遅延が許容できない重大通知。メイン障害時の代替経路 (Resend 直送フォールバック) の詳細は [index.md § 7.2 / § 13.2.2](#) を参照。

1.4.7 3.7 メール本文の表示義務（特定電子メール法）

メール本文末尾には `COMPANY_LEGAL_NAME / UNSUBSCRIBE_URL` 等の表示義務情報を含める（テンプレート側で必須）。

1.5 4. 関連設計

種別	参照先
要件	../01_要件定義/index.md
基本設計	../02_基本設計/index.md
メッセージ一覧	../02_基本設計/06_メッセージ一覧.md
運用設計	../04_運用設計/index.md
将来対応	../05_future/index.md

種別	参照先
関連 DD	DD11_ 暗号化・ 管理.md / DD14_ バッチ・ 非同期処理.md / DD10_ 監査ログ書込・ 完全性検証.md

1.6 5. テスト観点

AC ID	テスト ID	テスト方式	テストファイル
AC-021	it-notification-001	Integration	workers/queue-consumer/test/integration/notification.test.ts

1.6.1 5.1 その他観点

観点	内容
単体	ResendEmailProvider.send の HTTP エラー / Idempotency-Key 付与
結合	Webhook 受信 → 署名検証 → idempotency → 8 イベント分岐
異常系	soft bounce 5 連続 → permanent suppression / DLQ 投入後 R2 退避
境界値	TTL 600s 最大遅延ぴったり / 再試行回数 3 回ぴったり
性能	通知 Queue 投入 < 100ms (producer.send() のみ)
重要度	critical 通知の強制送信 (オプトアウト無視) の挙動確認

1.7 6. 未確定事項・確認事項

確認事項 ID	確認内容	優先度	ステータス
-	v1.0 リリース時点で全項目確定済み	低	確認済